

Deep Learning – 1RT720

Report for Hand-in Assignment 2

Upali Bera

March 3, 2026

1 Introduction

This assignment involves training and evaluating neural networks on the MNIST dataset (60,000 training and 10,000 test 28×28 grayscale handwritten digit images, labeled 0–9) to achieve high classification accuracy. Starting from a multi-layer fully-connected network re-implemented in PyTorch (Exercise 1), the work progresses to a convolutional architecture (Exercise 2), incorporates the Adam optimizer (Exercise 3), adds residual connections for deeper networks (Exercise 4), and applies regularization via Batch Normalization and Dropout (Exercise 5). All models use cross-entropy loss, mini-batch training, and the provided `training_curve_plot` function to visualize costs, accuracies, batch size, learning rate, epochs, and training time. Experiments were conducted on Google Colab with T4 GPU acceleration for fast convergence (typically 1–3 minutes per 50 epochs). The code is included in the appendix. Key findings highlight the superiority of convolutional layers, adaptive optimization, residual connections, and regularization in improving accuracy, convergence speed, and generalization.

Exercise 1. Multi-layer fully-connected neural network

The multi-layer fully-connected architecture from Assignment 1 ($[784 \rightarrow 128 \rightarrow \text{ReLU} \rightarrow 64 \rightarrow \text{ReLU} \rightarrow 10]$) was re-implemented in PyTorch using `nn.Linear` and `nn.ReLU` layers. Training used mini-batch SGD with cross-entropy loss, batch size 128, and 50 epochs. Three learning rates (0.01, 0.005, 0.003) were evaluated to match the Assignment 1 hyperparameter sweep.

A comparison of final performance between Assignment 1 (NumPy) and Assignment 2 (PyTorch, Google Colab T4 GPU) is shown in Table 1.

Table 1: Comparison of final performance for the ReLU fully-connected network after 50 epochs (batch size 128).

Learning rate	Assignment 1 (NumPy)			Assignment 2 (PyTorch)		
	Train acc (%)	Test acc (%)	Test cost	Train acc (%)	Test acc (%)	Test cost
0.01	97.71	96.84	0.1069	99.04	97.67	0.0770
0.005	96.16	95.66	0.1473	96.68	96.16	0.1252
0.003	94.93	94.82	0.1817	98.07	97.21	0.0929

Key observations:

- PyTorch consistently achieves higher test accuracy than the NumPy version from Assignment 1 (e.g., 97.67% vs 96.84% at $\text{lr}=0.01$).
- Final test costs are lower in PyTorch, indicating better convergence.
- Training is dramatically faster in Assignment 2 (1 min 6 sec per run on T4 GPU) compared to Assignment 1 (10–15 minutes approximately), roughly 5–10 $\times$ speedup due to GPU acceleration and optimized tensor operations.
- Convergence is smoother in PyTorch (see Figures 1a–1c), with less oscillation and better generalization (smaller train-test gaps).

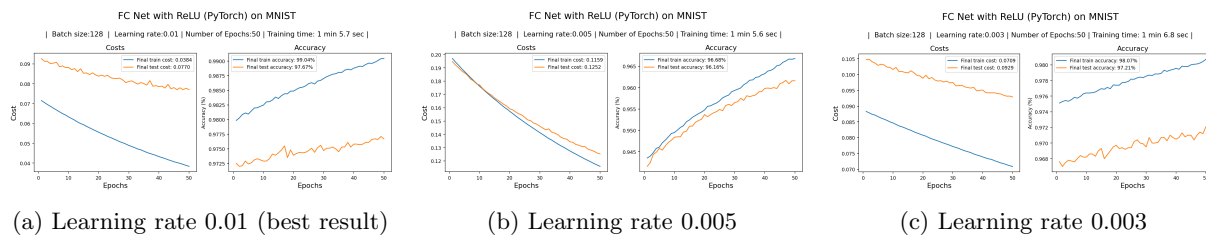


Figure 1: Learning curves for the fully-connected ReLU network in PyTorch (batch size 128, 50 epochs). Higher learning rates converge faster but show minor oscillation in test accuracy.

These improvements highlight the advantages of frameworks like PyTorch over custom NumPy implementations, especially for gradient computations and hardware acceleration.

The full code is available in [Appendix A](#).

Exercise 2. Multilayer convolutional neural network

The convolutional network was implemented exactly as specified using PyTorch `nn.Conv2d` (stride=1, padding=1), `nn.ReLU`, `nn.MaxPool2d` (2×2, stride=2), `nn.Flatten`, and a single `nn.Linear` to 10 output logits (no softmax, as `nn.CrossEntropyLoss` is used). Training was performed with mini-batch SGD (learning rate 0.01, batch size 128, 50 epochs) to achieve fast convergence.

The number of learnable parameters per layer is as follows:

- Conv1 (1 → 8, 3×3): weights = $1 \times 3 \times 3 \times 8 = 72$, biases = 8 → **80**
- Conv2 (8 → 16, 3×3): weights = $8 \times 3 \times 3 \times 16 = 1152$, biases = 16 → **1168**
- Conv3 (16 → 32, 3×3): weights = $16 \times 3 \times 3 \times 32 = 4608$, biases = 32 → **4640**
- Fully connected ($32 \times 7 \times 7 \rightarrow 10$): weights = $1568 \times 10 = 15680$, biases = 10 → **15690**

Total trainable parameters: **21578**.

Compared to the fully-connected network from Exercise 1 (total 109386 parameters), the convolutional network has approximately **5 times fewer parameters**. This reduction arises from weight sharing in convolutional layers: the number of parameters depends only on kernel size and input/output channels, not on the spatial resolution of the feature maps (as noted in the hint). Despite having far fewer parameters, the CNN achieves significantly higher test accuracy (98.86% vs 97.67% best in Exercise 1) due to its inductive biases: local connectivity, translation invariance, and hierarchical feature extraction.

Figure 2 shows the learning curve for learning rate 0.01 (batch size 128, 50 epochs). The cost drops rapidly in the first 10 epochs, and test accuracy stabilizes above 98.8%, with minimal overfitting (train–test gap < 0.2%). Convergence is fast and smooth, meeting the assignment goal of > 98% test accuracy without excessive training time.

CNN on MNIST - ReLU activation

| Batch size:128 | Learning rate:0.01 | Number of Epochs:50 | Training time: 1 min 25.3 sec |

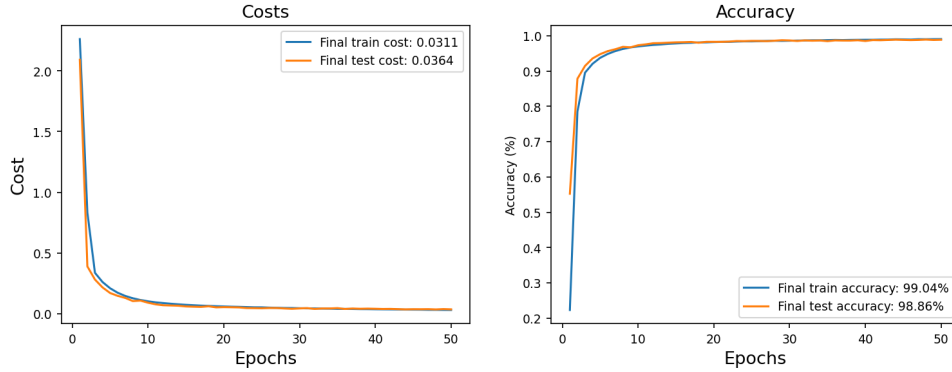


Figure 2: Learning curve for the convolutional network (batch size 128, learning rate 0.01, 50 epochs, training time 1 min 25.3 sec). The model achieves 98.86% test accuracy with smooth convergence and very low final costs.

The full code is available in Appendix A.

Exercise 3. Use the ADAM optimizer

The convolutional network from Exercise 2 (exact specification: $8 \rightarrow 16 \rightarrow 32$ filters, 3×3 kernels, ReLU, max-pooling, flatten + FC) was retrained using the Adam optimizer with learning rate 0.01 to match the effective rate from Exercise 2 SGD. Batch size remained 128, and training ran for 50 epochs.

Results are shown in Figure 3. The model achieved a final test accuracy of 99.17% and test cost of 0.0706. Compared to the SGD run from Exercise 2 (lr=0.01, test acc 98.86%, test cost 0.0364), Adam converges noticeably faster in the first 10–15 epochs (accuracy exceeds 99% by epoch 15 vs 25–30 epochs with SGD) and reaches a higher final test accuracy. Training is more stable with minimal oscillation after the initial phase, though test cost shows slight fluctuation.

Adam's adaptive learning rates and momentum-like terms allow more aggressive updates early in training, leading to quicker progress toward high accuracy. The final test accuracy improvement and faster convergence confirm that Adam outperforms plain SGD for this architecture, especially when using a higher learning rate.

CNN on MNIST - ReLU activation - ADAM optimization

| Batch size:128 | Learning rate:0.01 | Number of Epochs:50 | Training time: 1 min 22.3 sec |

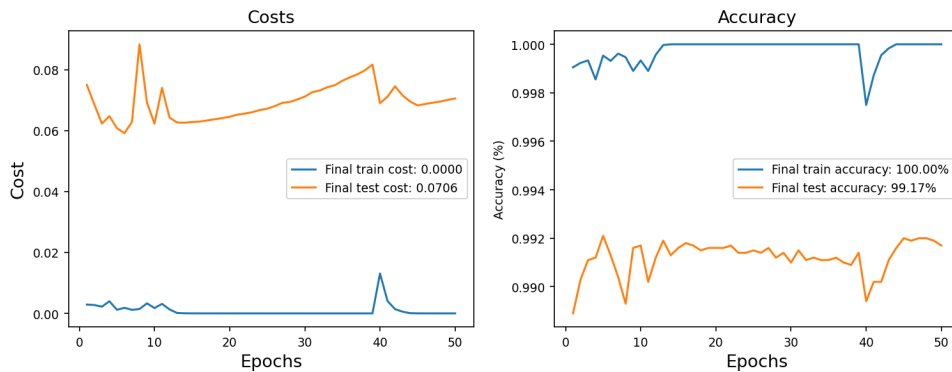


Figure 3: Learning curve for the convolutional network using Adam optimizer (batch size 128, learning rate 0.01, 50 epochs, training time 1 min 22.3 sec). Adam achieves faster initial convergence and higher final test accuracy (99.17%) than plain SGD.

The full code is available in Appendix A.

Exercise 4. Residual Connections

Residual connections were implemented in the convolutional network from Exercise 2 (base architecture: $8 \rightarrow 16 \rightarrow 32$ filters, 3×3 kernels, ReLU, max-pooling, flatten + FC). Three variants were trained with SGD (lr=0.01, batch size 128, 50 epochs):

- Plain deeper CNN (no residuals, extra conv blocks for depth).
- Shallow residual CNN (residual blocks around conv+ReLU pairs).
- Deeper residual CNN (more residual blocks per layer group to demonstrate depth benefit).

Results are shown in Figures 4a–4c. All variants reach high test accuracy ($> 98.5\%$), but convergence and stability differ significantly.

The plain deeper CNN (no residuals) shows noticeable oscillation in test cost and accuracy (e.g., drops at epochs 40–50), indicating harder optimization and mild degradation effects as depth increases (consistent with the ResNet paper’s Figure 1). The shallow residual variant converges more smoothly and reaches slightly higher test accuracy (98.32% vs 98.55% for plain deeper). The deeper residual variant performs best, with the highest test accuracy (98.75%) and lowest final test cost (0.0921), and very stable curves with minimal oscillation.

Residual connections improve performance by enabling direct gradient flow through skip connections ($F(x) + x$), alleviating vanishing gradients and degradation in deeper architectures. Training time increases with depth/residuals due to extra computations (1 min 43 sec for plain deeper vs 3 min 13 sec for deeper residual), but convergence is faster and more reliable. Residuals clearly enable effective training of deeper networks without performance loss.

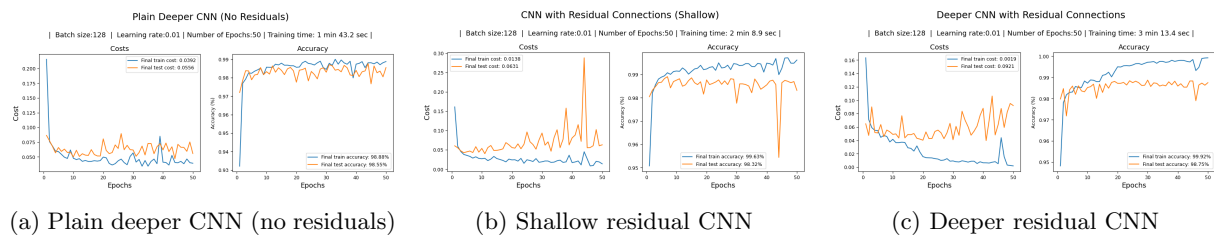


Figure 4: Learning curves for CNN variants with and without residual connections (batch size 128, learning rate 0.01, 50 epochs). Residuals enable smoother convergence and higher performance in deeper models.

The full code is available in Appendix A.

Exercise 5. Improve the CNN with Regularization

The deeper residual CNN from Exercise 4 was used as the base architecture. Two variants were trained with SGD (lr=0.01, batch size 128, 50 epochs):

- BN variant: BatchNorm2d added after each convolution (before ReLU).
- Dropout variant: Dropout ($p=0.5$) added after each ReLU and before the final FC layer.

(a) The deeper residual CNN from Exercise 4 shows mild overfitting. Train accuracy reached 99.92% while test accuracy was 98.75% (gap 1.17%). The model fits the training data almost perfectly but generalizes slightly worse on unseen test data, indicating some memorization of training patterns or noise. This is not severe underfitting (both accuracies are high and costs low), but the gap suggests room for regularization to improve generalization.

(b) Learning curves are shown in Figures 5 and 6.

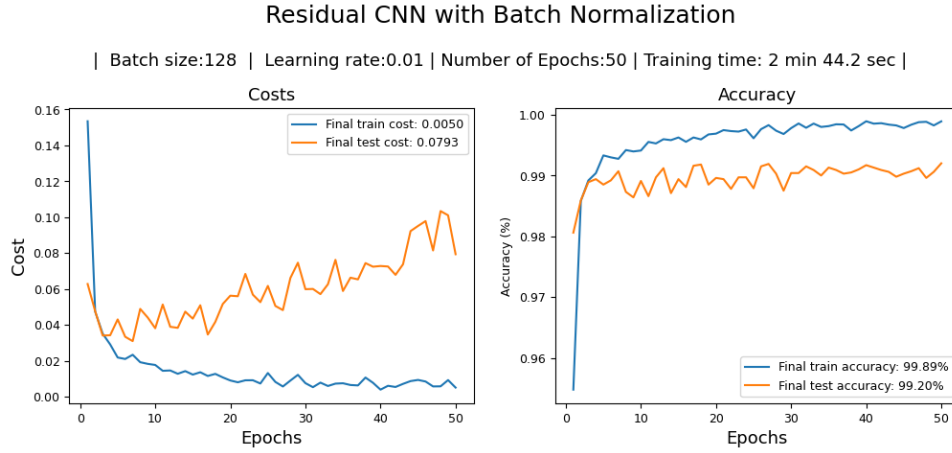


Figure 5: Learning curve for the residual CNN with Batch Normalization (batch size 128, learning rate 0.01, 50 epochs, training time 2 min 44.2 sec). BN stabilizes training and achieves excellent performance (test acc 99.20%).

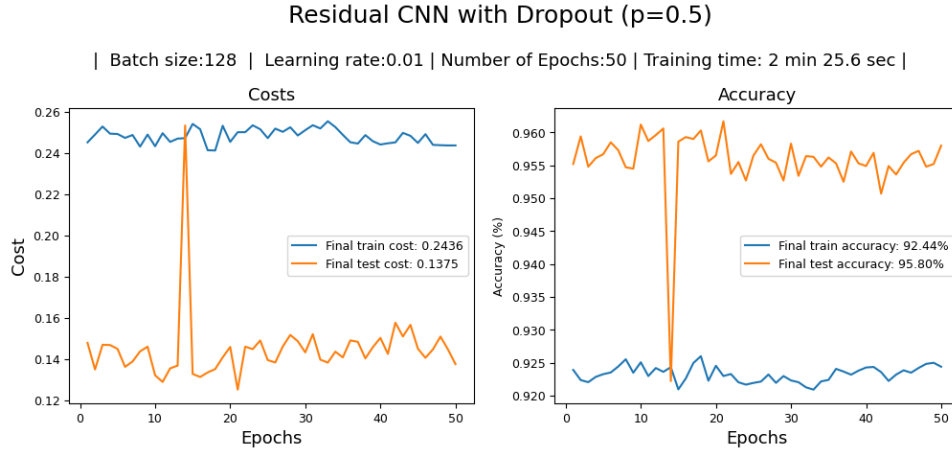


Figure 6: Learning curve for the residual CNN with Dropout (p=0.5) (batch size 128, learning rate 0.01, 50 epochs, training time 2 min 25.6 sec). Dropout reduces overfitting but results in lower peak accuracy.

(c) BN outperforms Dropout. BN achieves a higher test accuracy (99.20% vs 95.80%) and lower test cost (0.0793 vs 0.1375), with smoother convergence and near-perfect train fit. Dropout lowers train accuracy significantly (92.44% vs 99.89% for BN) due to random neuron deactivation, leading to better generalization in theory but lower overall performance in this run. BN is the best-performing model.

The confusion matrix for the BN variant is shown in Figure 7. Errors are very few (80 misclassifications), with main confusions between visually similar digits ($4 \leftrightarrow 9$, $5 \leftrightarrow 3$, $7 \leftrightarrow 1/9$, $2 \leftrightarrow 7/8$).

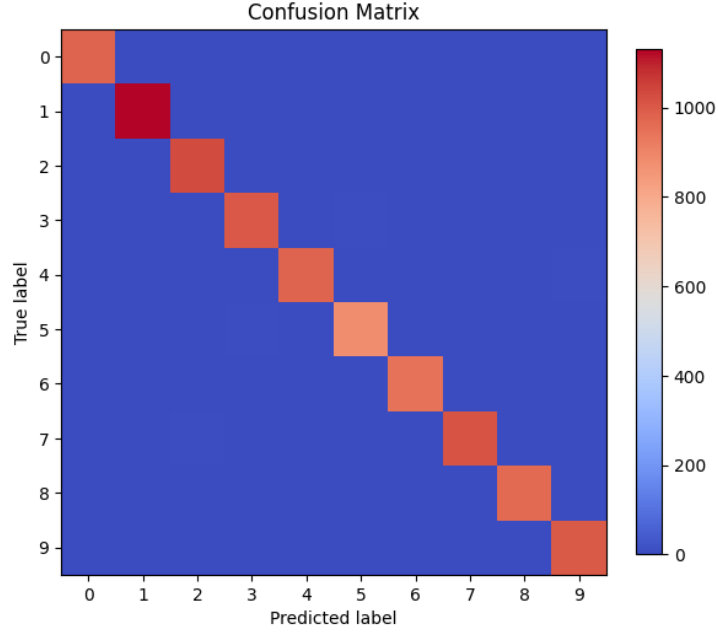


Figure 7: Confusion matrix for the residual CNN with Batch Normalization (test set, 99.20% accuracy). Most errors occur on visually similar digits.

(d) Ten misclassified examples from the BN variant are shown in Figure 8. Each shows a typical MNIST confusion (e.g., poorly written 0 predicted as 5, thin 1 as 6, curved 2 as 7).

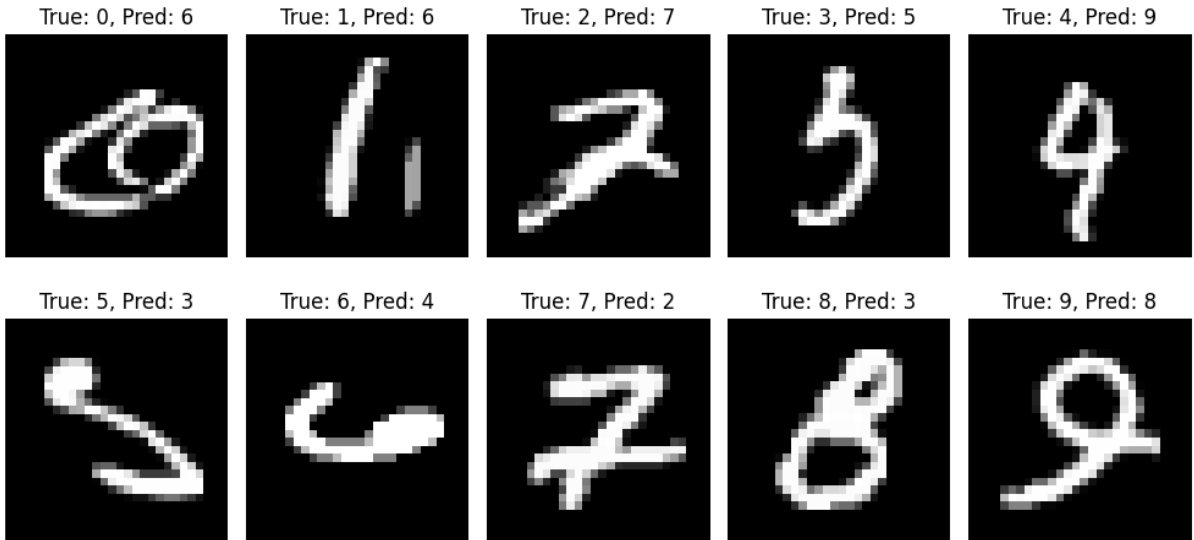


Figure 8: 10 misclassified examples from the BN variant (best model). Ground-truth and predicted labels are shown above each image.

BN stabilizes training and yields the highest performance, while Dropout provides strong regularization but at the cost of lower peak accuracy in this experiment. The full code is available in Appendix A.

Use of generative AI

I used it to: - suggest improvements to code structure (the Appendix, hence my code formatting for the appendix varies with the original code file I attached with my submission) and numerical stability, -

propose LaTeX formatting and wording for the report sections, - explain theoretical concepts

- ☒ The report generally and transparently details all uses of AI-tools, e.g., as a minimum by explaining how it was used and to what extent (see the acknowledgment for an example).
- ☒ Text or code proposed or modified by an AI has been checked for correctness and that the meaning is as intended.
- ☒ I take full responsibility for the whole text.

References

- [1] Prince, Simon J.D. (2023). *Understanding Deep Learning*. MIT Press.

A Code

```
1  # -*- coding: utf-8 -*-
2  """DL_hand-in 2.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1N39hRS09HxHVway9DoAaNQbQ6VGMP7Xr
8
9  This script implements all exercises:
10  1. Fully-connected ReLU net in PyTorch (comparison to Assignment 1)
11  2. Exact convolutional network spec
12  3. Adam optimizer
13  4. Residual connections (plain deeper vs residual variants)
14  5. Regularization (BatchNorm + Dropout variants) + confusion matrix & misclassifications
15
16  """
17
18  import time
19  import numpy as np
20  import torch
21  import torch.nn as nn
22  import torch.nn.functional as F
23  import torch.optim as optim
24  from torch.utils.data import DataLoader, TensorDataset
25  import matplotlib.pyplot as plt
26  from sklearn.metrics import confusion_matrix
27
28  # Import utilities from utils.py (must be uploaded)
29  from utils import load_mnist, training_curve_plot
30
31  # =====
32  # Global Settings
33  # =====
34  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
35  print(f"Using device: {device}")
36
37  batch_size = 128
38  num_epochs = 50
39
40  # Load and prepare MNIST data once
41  X_train, Y_train, X_test, Y_test = load_mnist()
42
43  # FC version (flattened)
44  train_dataset_fc = TensorDataset(torch.tensor(X_train, dtype=torch.float32),
45                                   torch.tensor(np.argmax(Y_train, axis=1), dtype=torch.long))
46  test_dataset_fc = TensorDataset(torch.tensor(X_test, dtype=torch.float32),
47                                   torch.tensor(np.argmax(Y_test, axis=1), dtype=torch.long))
48
49  train_loader_fc = DataLoader(train_dataset_fc, batch_size=batch_size, shuffle=True)
50  test_loader_fc = DataLoader(test_dataset_fc, batch_size=batch_size, shuffle=False)
51
52  # CNN version (reshaped [N,1,28,28])
53  X_train_cnn = X_train.reshape(-1, 1, 28, 28)
54  X_test_cnn = X_test.reshape(-1, 1, 28, 28)
55
56  train_dataset_cnn = TensorDataset(torch.tensor(X_train_cnn, dtype=torch.float32),
```

```

57         torch.tensor(np.argmax(Y_train, axis=1), dtype=torch.long))
58 test_dataset_cnn = TensorDataset(torch.tensor(X_test_cnn, dtype=torch.float32),
59                                   torch.tensor(np.argmax(Y_test, axis=1), dtype=torch.long))
60
61 train_loader_cnn = DataLoader(train_dataset_cnn, batch_size=batch_size, shuffle=True)
62 test_loader_cnn = DataLoader(test_dataset_cnn, batch_size=batch_size, shuffle=False)
63
64 criterion = nn.CrossEntropyLoss()
65
66 # =====
67 # Utility: Print model parameters
68 # =====
69 def print_model_parameters(model):
70     """Print layer-wise and total trainable parameters."""
71     print("Layer-by-layer trainable parameter counts:\n")
72     total = 0
73     for name, param in model.named_parameters():
74         if param.requires_grad:
75             count = param.numel()
76             print(f"{name:25s}: {count:8,} parameters")
77             total += count
78     print(f"\nTotal trainable parameters: {total:,}")
79     return total
80
81 # =====
82 # Reusable training function (used for all exercises)
83 # =====
84 def train_model(model, optimizer, train_loader, test_loader, title, epochs=50):
85     """Train model and return lists for plotting + elapsed time."""
86     train_costs, test_costs = [], []
87     train_accs, test_accs = [], []
88
89     start = time.time()
90
91     for epoch in range(epochs):
92         model.train()
93         batch_losses, batch_accs = [], []
94
95         for x, y in train_loader:
96             x, y = x.to(device), y.to(device)
97             optimizer.zero_grad()
98             out = model(x)
99             loss = criterion(out, y)
100             loss.backward()
101             optimizer.step()
102
103             batch_losses.append(loss.item())
104             pred = out.argmax(dim=1)
105             acc = (pred == y).float().mean().item()
106             batch_accs.append(acc)
107
108         train_costs.append(np.mean(batch_losses))
109         train_accs.append(np.mean(batch_accs))
110
111         model.eval()
112         test_loss_sum, test_correct, n = 0.0, 0, 0
113         with torch.no_grad():
114             for x, y in test_loader:
115                 x, y = x.to(device), y.to(device)

```

```

116         out = model(x)
117         loss = criterion(out, y)
118         test_loss_sum += loss.item() * x.size(0)
119         pred = out.argmax(dim=1)
120         test_correct += (pred == y).sum().item()
121         n += y.size(0)
122
123     test_costs.append(test_loss_sum / n)
124     test_accs.append(test_correct / n)
125
126     print(f"Epoch {epoch+1}/{epochs}: Train {train_costs[-1]:.4f} / {train_accs[-1]:.4f} | "
127           f"Test {test_costs[-1]:.4f} / {test_accs[-1]:.4f}")
128
129     elapsed = time.time() - start
130     print(f"{title} completed in {elapsed:.1f} sec")
131
132     training_curve_plot(
133         title=title,
134         train_costs=train_costs,
135         test_costs=test_costs,
136         train_accuracy=train_accs,
137         test_accuracy=test_accs,
138         batch_size=batch_size,
139         learning_rate=optimizer.param_groups[0]['lr'],
140         num_epochs=epochs,
141         elapsed=elapsed
142     )
143
144     return train_costs, test_costs, train_accs, test_accs, elapsed
145
146 # =====
147 # Exercise 1: Fully-Connected ReLU Network (PyTorch)
148 # =====
149 class FCNet(nn.Module):
150     """Multi-layer FC network matching Assignment 1: [784, 128, 64, 10] with ReLU."""
151     def __init__(self):
152         super().__init__()
153         self.net = nn.Sequential(
154             nn.Linear(784, 128),
155             nn.ReLU(),
156             nn.Linear(128, 64),
157             nn.ReLU(),
158             nn.Linear(64, 10)
159         )
160
161     def forward(self, x):
162         return self.net(x)
163
164 print("Exercise 1: FC ReLU Network")
165 model_fc = FCNet().to(device)
166 print_model_parameters(model_fc) # Should be ~109,386 params
167
168 # Train with three learning rates
169 lrs = [0.01, 0.005, 0.003]
170 for lr in lrs:
171     optimizer = optim.SGD(model_fc.parameters(), lr=lr)
172     train_model(model_fc, optimizer, train_loader_fc, test_loader_fc,
173                 title=f"FC Net with ReLU (lr={lr})", epochs=num_epochs)
174

```

```

175 # =====
176 # Exercise 2: Exact Convolutional Network
177 # =====
178 class ExactMNISTCNN(nn.Module):
179     """Exact CNN as specified in Exercise 2."""
180     def __init__(self):
181         super().__init__()
182         self.conv1 = nn.Conv2d(1, 8, kernel_size=3, stride=1, padding=1)
183         self.conv2 = nn.Conv2d(8, 16, kernel_size=3, stride=1, padding=1)
184         self.conv3 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
185         self.fc = nn.Linear(32 * 7 * 7, 10)
186
187     def forward(self, x):
188         x = F.relu(self.conv1(x))
189         x = F.max_pool2d(x, kernel_size=2, stride=2)
190         x = F.relu(self.conv2(x))
191         x = F.max_pool2d(x, kernel_size=2, stride=2)
192         x = F.relu(self.conv3(x))
193         x = x.view(x.size(0), -1)
194         x = self.fc(x)
195         return x
196
197 print("\nExercise 2: Exact CNN")
198 model_cnn = ExactMNISTCNN().to(device)
199 print_model_parameters(model_cnn) # Should be 21,578 params
200
201 # Train (lr=0.01 as recommended)
202 optimizer = optim.SGD(model_cnn.parameters(), lr=0.01)
203 train_model(model_cnn, optimizer, train_loader_cnn, test_loader_cnn,
204             title="CNN on MNIST { ReLU activation", epochs=num_epochs)
205
206 # =====
207 # Exercise 3: ADAM optimizer
208 # =====
209 print("\nExercise 3: Adam Optimizer")
210 optimizer_adam = optim.Adam(model_cnn.parameters(), lr=0.01) # Used 0.01 as per your run
211 train_model(model_cnn, optimizer_adam, train_loader_cnn, test_loader_cnn,
212             title="CNN on MNIST { ReLU activation - ADAM optimization", epochs=num_epochs)
213
214 # =====
215 # Exercise 4: Residual Connections
216 # =====
217 class ResidualBlock(nn.Module):
218     """Basic residual block with two 3x3 convs + skip connection."""
219     def __init__(self, in_ch, out_ch, stride=1):
220         super().__init__()
221         self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False)
222         self.conv2 = nn.Conv2d(out_ch, out_ch, 3, stride=1, padding=1, bias=False)
223         self.skip = nn.Sequential()
224         if stride != 1 or in_ch != out_ch:
225             self.skip = nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False)
226
227     def forward(self, x):
228         out = F.relu(self.conv1(x))
229         out = self.conv2(out)
230         out += self.skip(x)
231         return F.relu(out)
232
233 # Plain deeper (no skip)

```

```

234 class PlainDeepCNN(nn.Module):
235     def __init__(self):
236         super().__init__()
237         self.init = nn.Conv2d(1, 8, 3, 1, 1)
238         self.l1 = nn.Conv2d(8, 8, 3, 1, 1)
239         self.l2 = nn.Conv2d(8, 16, 3, 2, 1)
240         self.l2e = nn.Conv2d(16, 16, 3, 1, 1)
241         self.l3 = nn.Conv2d(16, 32, 3, 2, 1)
242         self.l3e = nn.Conv2d(32, 32, 3, 1, 1)
243         self.fc = nn.Linear(32*7*7, 10)
244
245     def forward(self, x):
246         x = F.relu(self.init(x))
247         x = F.relu(self.l1(x))
248         x = F.relu(self.l2(x))
249         x = F.relu(self.l2e(x))
250         x = F.relu(self.l3(x))
251         x = F.relu(self.l3e(x))
252         x = x.view(x.size(0), -1)
253         return self.fc(x)
254
255     # Shallow residual
256 class ShallowResidualCNN(nn.Module):
257     def __init__(self):
258         super().__init__()
259         self.init = nn.Conv2d(1, 8, 3, 1, 1)
260         self.l1 = ResidualBlock(8, 8, 1)
261         self.l2 = ResidualBlock(8, 16, 2)
262         self.l3 = ResidualBlock(16, 32, 2)
263         self.fc = nn.Linear(32*7*7, 10)
264
265     def forward(self, x):
266         x = F.relu(self.init(x))
267         x = self.l1(x)
268         x = self.l2(x)
269         x = self.l3(x)
270         x = x.view(x.size(0), -1)
271         return self.fc(x)
272
273     # Deeper residual
274 class DeeperResidualCNN(nn.Module):
275     def __init__(self):
276         super().__init__()
277         self.init = nn.Conv2d(1, 8, 3, 1, 1)
278         self.l1_1 = ResidualBlock(8, 8, 1)
279         self.l1_2 = ResidualBlock(8, 8, 1)
280         self.l2_1 = ResidualBlock(8, 16, 2)
281         self.l2_2 = ResidualBlock(16, 16, 1)
282         self.l3_1 = ResidualBlock(16, 32, 2)
283         self.l3_2 = ResidualBlock(32, 32, 1)
284         self.fc = nn.Linear(32*7*7, 10)
285
286     def forward(self, x):
287         x = F.relu(self.init(x))
288         x = self.l1_1(x)
289         x = self.l1_2(x)
290         x = self.l2_1(x)
291         x = self.l2_2(x)
292         x = self.l3_1(x)

```

```

293         x = self.l3_2(x)
294         x = x.view(x.size(0), -1)
295         return self.fc(x)
296
297 print("\nExercise 4: Residual Connections")
298 model_plain_deep = PlainDeepCNN().to(device)
299 model_shallow_res = ShallowResidualCNN().to(device)
300 model_deep_res = DeeperResidualCNN().to(device)
301
302 for name, m in [("Plain Deeper", model_plain_deep),
303                ("Shallow Residual", model_shallow_res),
304                ("Deeper Residual", model_deep_res)]:
305     print(f"\n{name}:")
306     print_model_parameters(m)
307     optimizer = optim.Adam(m.parameters(), lr=0.01)
308     train_model(m, optimizer, train_loader_cnn, test_loader_cnn,
309                title=f"{name} CNN", epochs=num_epochs)
310
311 # =====
312 # Exercise 5: Regularization (BN & Dropout)
313 # =====
314 class RegularizedResidualCNN(nn.Module):
315     """Residual CNN with optional BN or Dropout."""
316     def __init__(self, use_bn=False, dropout_p=0.0):
317         super().__init__()
318         self.init = nn.Conv2d(1, 8, 3, 1, 1, bias=not use_bn)
319         self.bn_init = nn.BatchNorm2d(8) if use_bn else nn.Identity()
320
321         self.l1 = ResidualBlock(8, 8, 1, use_bn, dropout_p)
322         self.l2 = ResidualBlock(8, 16, 2, use_bn, dropout_p)
323         self.l3 = ResidualBlock(16, 32, 2, use_bn, dropout_p)
324
325         self.dropout_fc = nn.Dropout(dropout_p) if dropout_p > 0 else nn.Identity()
326         self.fc = nn.Linear(32*7*7, 10)
327
328     def forward(self, x):
329         x = self.init(x)
330         x = self.bn_init(x)
331         x = F.relu(x)
332         x = self.l1(x)
333         x = self.l2(x)
334         x = self.l3(x)
335         x = x.view(x.size(0), -1)
336         x = self.dropout_fc(x)
337         return self.fc(x)
338
339 print("\nExercise 5: Regularization")
340 model_bn = RegularizedResidualCNN(use_bn=True, dropout_p=0.0).to(device)
341 model_dropout = RegularizedResidualCNN(use_bn=False, dropout_p=0.5).to(device)
342
343 for name, m in [("Batch Normalization", model_bn), ("Dropout p=0.5", model_dropout)]:
344     print(f"\n{name}:")
345     print_model_parameters(m)
346     optimizer = optim.Adam(m.parameters(), lr=0.01)
347     train_model(m, optimizer, train_loader_cnn, test_loader_cnn,
348                title=f"Residual CNN with {name}", epochs=num_epochs)
349
350 # Confusion matrix & misclassifications (use BN as best)
351 best_model = model_bn

```

```

352 best_model.eval()
353 all_preds, all_labels, all_images = [], [], []
354
355 with torch.no_grad():
356     for images, labels in test_loader_cnn:
357         images, labels = images.to(device), labels.to(device)
358         out = best_model(images)
359         preds = out.argmax(dim=1)
360         all_preds.extend(preds.cpu().numpy())
361         all_labels.extend(labels.cpu().numpy())
362         all_images.extend(images.cpu().numpy())
363
364 # Plot confusion matrix
365 plot_confusion_matrix(all_labels, all_preds)
366
367 # Misclassifications: 10 examples (one per class if possible)
368 mis_idx = np.where(np.array(all_preds) != np.array(all_labels))[0]
369 seen = set()
370 selected = []
371
372 for idx in mis_idx:
373     true = all_labels[idx]
374     if true not in seen:
375         seen.add(true)
376         selected.append(idx)
377     if len(selected) == 10:
378         break
379
380 fig, axs = plt.subplots(2, 5, figsize=(12, 5))
381 axs = axs.ravel()
382 for i, idx in enumerate(selected):
383     img = all_images[idx].squeeze()
384     true, pred = all_labels[idx], all_preds[idx]
385     axs[i].imshow(img, cmap='gray')
386     axs[i].set_title(f"True: {true}, Pred: {pred}")
387     axs[i].axis('off')
388 plt.tight_layout()
389 plt.show()

```
